PG Merchant Compliance API Documentation

Payment Gateway Merchant Compliance API Documentation

Version: 2.1
Last Updated: November 2026
Service: gateway-merchant

Table of Contents

1. Overview
2. Authentication
3. Idempotency
4. Optimistic Concurrency Control
5. Business Types & Endpoint Flows
6. API Endpoints Reference
7. Starter Business Flow
8. Registered Business Flow
9. Document Upload
10. Error Handling
11. FAQ (Frontend Integration Questions)
12. Completion Checks

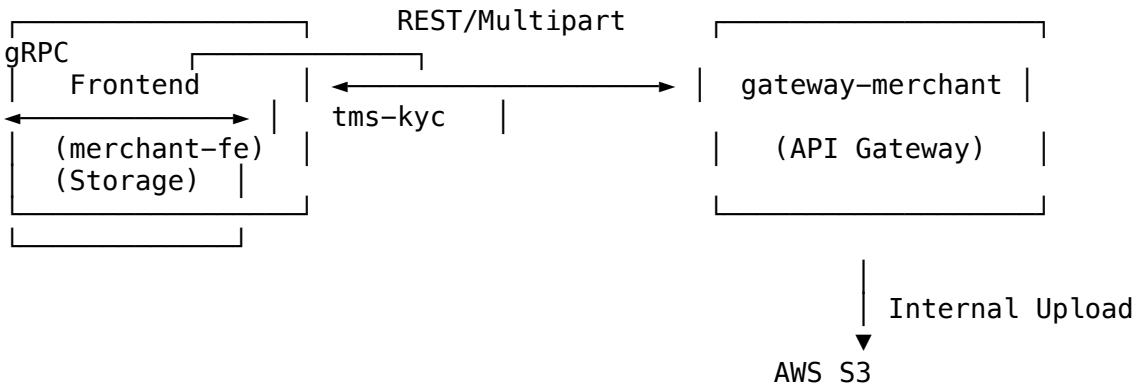
---

1. Overview

The Payment Gateway Merchant Compliance API enables merchants to complete their KYC (Know Your Customer) compliance requirements for payment gateway access. The API supports two business types with different requirements and **different endpoint flows**:

- **Starter Business**: For unregistered businesses/sole proprietors (single owner)
- **Registered Business**: For CAC-registered companies (multiple shareholders)

Architecture



Key Design: Multipart Form Upload

Documents are submitted **inline with form data** at each step using `multipart/form-data`. This reduces the total API calls to a maximum of **2 calls per step** (one save + optional submit), regardless of how many documents are attached.

Base URL

```
Production: https://gm.irpay.dev
Staging:    https://staging-gm.irpay.dev
```

---

2. Authentication

All endpoints (except `/compliance/schema`) require Bearer token authentication.

Required Headers

```
Authorization: Bearer <access_token>
```

Obtaining a Token

```
POST /auth/login
Content-Type: application/json

{
  "email": "merchant@example.com",
  "password": "secure_password"
}
```

Response:

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
```

```
    "expires_in": 3600
  }
}
```

---

3. Idempotency

What is Idempotency?

Idempotency ensures that if a request fails (network timeout, server error) and you retry it with the **same idempotency key**, the server will: - Return the same response as the first successful attempt - NOT create duplicate records or perform the action twice

How to Use

All mutating endpoints (POST, PUT, PATCH, DELETE) require an `Idempotency-Key` header:

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

Rules for Idempotency Keys

Rule	Description
Format	UUID v4 or alphanumeric string (max 128 chars)
Uniqueness	Each distinct operation needs a unique key
Scope	Keys are scoped per merchant
TTL	Keys expire 24 hours after first use
Retry	Same key + same request = returns cached response
Conflict	Same key + different request = HTTP 409 error

---

4. Optimistic Concurrency Control

How It Works

1. **GET** compliance → response includes `version: 5`
2. **PUT** with header `X-Expected-Version: 5`
3. If current version is still 5 → update succeeds, version becomes 6
4. If current version changed → HTTP 409 Conflict (refresh and retry)

Required Header

```
X-Expected-Version: 5
```

---

5. Business Types & Endpoint Flows

⚠️ Important: Different Endpoints for Different Business Types

Starter and Registered businesses use **different endpoints** for some steps. Using the wrong endpoint will result in validation errors.

Endpoint Flow Summary

Starter Business

Step	Endpoint	Documents
Profile	`PUT /compliance/profile`	None
Contact	`PUT /compliance/contact`	None
Owners	`PUT /compliance/owners`	`id_document_id` (ID), `proof_of_address_document_id` (address proof)
Bank Accounts	`PUT /compliance/bank-accounts`	None
Agreement	`POST /compliance/submit`	None

Registered Business

Step	Endpoint	Documents
Business Details	`PUT /compliance/business/details`	`business_address_docum` (address proof)
Registration	`PUT /compliance/business/registration`	None
Shareholders	`PUT /compliance/business/shareholders`	Per shareholder: `id_docume` `proof_of_address_docum`
Bank Accounts	`PUT /compliance/bank-accounts`	None
Agreement	`POST /compliance/submit`	None

Document Requirements

Starter Business: - Owner ID document (NIN, BVN, Passport, or Driver's License) - Owner proof of address (Utility bill ≤ 3 months old)

Registered Business: - Business proof of address - Per shareholder: ID document + proof of address

Valid ID Document Types

The following values are accepted for `id_document_type`: - `nin` - National Identification Number - `bvn` - Bank Verification Number - `international_passport` - International Passport - `drivers_license` - Driver's License

---

6. API Endpoints Reference

Public Endpoint

Method	Endpoint	Description
GET	`/compliance/schema?business_type=X`	Get form schema (no auth required)

Common Compliance Endpoints

Method	Endpoint	Description
POST	`/compliance/start`	Start new compliance
GET	`/compliance`	Get current compliance state
PUT	`/compliance/bank-accounts`	Save bank accounts
POST	`/compliance/submit`	Submit for review

Starter Business Endpoints

Method	Endpoint	Content-Type	Description
PUT	`/compliance/profile`	`application/json`	Save profile
PUT	`/compliance/contact`	`application/json`	Save contact
PUT	`/compliance/owners`	`multipart/form-data`	Save owners + documents

Registered Business Endpoints

Method	Endpoint	Content-Type	Descripti
PUT	`/compliance/business/details`	`multipart/form-data`	Save business details + address proof
PUT	`/compliance/business/registration`	`application/json`	Save RC number & TIN

Method	Endpoint	Content-Type	Descriptio
PUT	`/compliance/business/shareholders`	`multipart/form-data`	Save shareholde + document

---

7. Starter Business Flow

Complete Step-by-Step

```
                    STARTER BUSINESS FLOW
```

```
STEP 1: START COMPLIANCE
```

```
POST /compliance/start
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-1>

Response:
{
  "code": 200,
  "data": {
    "compliance_id": 100,
    "status": "draft",
    "version": 1
  }
}


STEP 2: SAVE PROFILE
```

```
PUT /compliance/profile
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-2>
  X-Expected-Version: 1
  Content-Type: application/json

Body:
{
  "country": "NG",
  "industry": "retail",
  "category": "fashion",
  "trading_name": "ACME Store",
  "sales_volume": "100000",
  "business_type": "starter",
  "business_description": "Fashion retail business"
}

Response: { "data": { "version": 2, "profile_complete": true } }
```

```
STEP 3: SAVE CONTACT
─────────────────────────────────────────────────────────────────────────────

PUT /compliance/contact
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-3>
  X-Expected-Version: 2
  Content-Type: application/json

Body:
{
  "email": "contact@acme.com",
  "support_email": "support@acme.com",
  "dispute_email": "support@acme.com",
  "same_as_support_email": true,
  "phone": "08012345678",
  "address": "123 Victoria Island, Lagos",
  "state": "Lagos",
  "lga": "Eti-Osa",
  "online_links": [
    { "platform": "website", "url": "https://acme.com" }
  ]
}

Response: { "data": { "version": 3, "contact_complete": true } }


STEP 4: SAVE OWNERS WITH DOCUMENTS (Multipart)
─────────────────────────────────────────────────────────────────────────────

PUT /compliance/owners
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-4>
  X-Expected-Version: 3
  Content-Type: multipart/form-data

Form Parts:
┌─────────────────────────────────────────────────────────────────────────────
│ Part: "data" (JSON string)
│
│ {
│
│   "owners": [{
│
│     "full_name": "John Doe",
│
│     "date_of_birth": "15-03-1985",
│
│     "nationality": "NG",
│
│     "id_document_type": "nin",
│
│     "address": "456 Ikoyi, Lagos",
│
│     "same_as_business_address": false,
│
│     "state": "Lagos",
│
│     "lga": "Eti-Osa"
│
```

```
  }]

}
```

```
Part: "owner_0_identity" (file) → passport.pdf
```

```
Part: "owner_0_address" (file) → utility-bill.pdf
```

```
File part naming convention:
  - owner_{index}_identity  → ID document for owner at index
  - owner_{index}_address   → Proof of address for owner at index

Response: { "data": { "version": 4, "owners_complete": true } }
```

STEP 5: SAVE BANK ACCOUNT

```
PUT /compliance/bank-accounts
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-5>
  X-Expected-Version: 4
  Content-Type: application/json

Body:
{
  "bank_accounts": [{
    "bank_code": "044",
    "account_number": "0123456789",
    "account_name": "John Doe"
  }]
}

Response: { "data": { "version": 5, "bank_accounts_complete": true } }
```

STEP 6: SUBMIT FOR REVIEW

```
POST /compliance/submit
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid-6>
  X-Expected-Version: 5
  Content-Type: application/json

Body:
{
  "agreement_accepted": true
}

Response: { "data": { "version": 6, "status": "submitted",
"agreement_complete": true } }
```

STARTER FLOW COMPLETE!

```
Total API calls: 6
```

---

8. Registered Business Flow

Complete Step-by-Step

```
                    REGISTERED BUSINESS FLOW
```

```
STEP 1: START COMPLIANCE
```

```
POST /compliance/start
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>

Response: { "data": { "compliance_id": 200, "version": 1 } }


STEP 2: SAVE BUSINESS DETAILS WITH ADDRESS PROOF (Multipart)
```

```
PUT /compliance/business/details      ← NOTE: Different endpoint!
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Expected-Version: 1
  Content-Type: multipart/form-data

Form Parts:
```

```
  Part: "data" (JSON string)

  {

    "country": "NG",

    "industry": "technology",

    "category": "fintech",

    "trading_name": "TechCorp Limited",

    "sales_volume": "5000000",

    "business_type": "registered",

    "business_description": "Financial technology solutions",

    "phone": "09012345678",

    "address": "Plot 123 Tech Hub, Lekki",

    "support_email": "support@techcorp.ng",
```

```
  "dispute_email": "disputes@techcorp.ng",
  "state": "Lagos",
  "lga": "Eti-Osa",
  "online_links": [
    { "platform": "website", "url": "https://techcorp.ng" }
  ]
}
```

```
Part: "business_address_proof" (file) → office-lease.pdf
```

```
Response: { "data": { "version": 2, "profile_complete": true,
"contact_complete": true } }
```

```
STEP 3: SAVE REGISTRATION DETAILS
```

```
PUT /compliance/business/registration      ← NOTE: Different endpoint!
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Expected-Version: 2
  Content-Type: application/json

Body:
{
  "registration_number": "RC1234567",
  "tax_identification_number": "12345678-0001"
}

Response: { "data": { "version": 3, "registration_complete": true } }
```

```
STEP 4: SAVE SHAREHOLDERS WITH DOCUMENTS (Multipart)
```

```
PUT /compliance/business/shareholders      ← NOTE: Different endpoint!
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Expected-Version: 3
  Content-Type: multipart/form-data

Form Parts:
```

```
Part: "data" (JSON string)
{
  "shareholders": [
    {
      "full_name": "Adaora Okafor",
```

```
      "date_of_birth": "20-05-1980",
      "nationality": "NG",
      "role": "director",
      "id_document_type": "international_passport",
      "address": "12 Executive Estate, Ikoyi",
      "same_as_business_address": false,
      "state": "Lagos",
      "lga": "Eti-Osa"
    },
    {
      "full_name": "Chidi Eze",
      "date_of_birth": "08-11-1975",
      "nationality": "NG",
      "role": "shareholder",
      "id_document_type": "nin",
      "address": "45 Business District, VI",
      "same_as_business_address": false,
      "state": "Lagos",
      "lga": "Eti-Osa"
    }
  ]
}
```

```
Part: "owner_0_identity" (file) → adaora-passport.pdf
```

```
Part: "owner_0_address" (file) → adaora-utility.pdf
```

```
Part: "owner_1_identity" (file) → chidi-nin.pdf
```

```
Part: "owner_1_address" (file) → chidi-utility.pdf
```

```
File part naming convention:
  - owner_{index}_identity  → ID document for shareholder at index
```

```
  - owner_{index}_address   → Proof of address for shareholder at
index

Response: { "data": { "version": 4, "owners_complete": true } }


STEP 5: SAVE BANK ACCOUNT
─────────────────────────────────────────────────────────────────

PUT /compliance/bank-accounts
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Expected-Version: 4
  Content-Type: application/json

Body:
{
  "bank_accounts": [{
    "bank_code": "058",
    "account_number": "1234567890",
    "account_name": "TECHCORP LIMITED"
  }]
}

Response: { "data": { "version": 5, "bank_accounts_complete": true } }


STEP 6: SUBMIT FOR REVIEW
─────────────────────────────────────────────────────────────────

POST /compliance/submit
Headers:
  Authorization: Bearer <token>
  Idempotency-Key: <uuid>
  X-Expected-Version: 5
  Content-Type: application/json

Body:
{
  "agreement_accepted": true
}

Response: { "data": { "version": 6, "status": "submitted",
"agreement_complete": true } }


═══════════════════════════════════════════════════════════════
                      REGISTERED FLOW COMPLETE!
                      Total API calls: 6
═══════════════════════════════════════════════════════════════
```

---

9. Document Upload

Multipart Upload Design

Documents are submitted **inline with form data** using `multipart/form-data`. This approach:

- **Reduces API calls**: One request per step, regardless of document count
- **Simplifies client code**: No separate presigned URL dance
- **Atomic operations**: Form data and files saved together or fail together

File Part Naming Convention

Endpoint	File Part Name	Description
`/compliance/business/details`	`business_address_proof`	Business address proof
`/compliance/owners`	`owner_{n}_identity`	Owner's ID document
`/compliance/owners`	`owner_{n}_address`	Owner's proof of address
`/compliance/business/shareholders`	`owner_{n}_identity`	Shareholder's ID document
`/compliance/business/shareholders`	`owner_{n}_address`	Shareholder's proof of address

What does `{n}` mean?

The `{n}` is the **0-based array index** matching the position in the JSON `owners` or `shareholders` array.

Example: If you submit 3 shareholders in the JSON:

```
{
  "shareholders": [
    { "full_name": "Alice" },   // index 0
    { "full_name": "Bob" },     // index 1
    { "full_name": "Charlie" }  // index 2
  ]
}
```

The file parts should be named:

```
owner_0_identity  → Alice's ID
owner_0_address   → Alice's proof of address
owner_1_identity  → Bob's ID
owner_1_address   → Bob's proof of address
owner_2_identity  → Charlie's ID
owner_2_address   → Charlie's proof of address
```

Supported File Types

Content Type	Extensions	Max Size
`application/pdf`	.pdf	1 MB
`image/jpeg`	.jpg, .jpeg	1 MB

Content Type	Extensions	Max Size
image/png	.png	1 MB

JavaScript Example

```
async function saveOwnersWithDocuments(owners, ownerFiles, token,
        version) {
  const formData = new FormData();

  // Add JSON data as a string part
  formData.append('data', JSON.stringify({ owners }));

  // Add files for each owner
  ownerFiles.forEach((files, index) => {
    if (files.identity) {
      formData.append(`owner_${index}_identity`, files.identity);
    }
    if (files.address) {
      formData.append(`owner_${index}_address`, files.address);
    }
  });

  const response = await fetch('/compliance/owners', {
    method: 'PUT',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Idempotency-Key': crypto.randomUUID(),
      'X-Expected-Version': version
      // Note: Don't set Content-Type - browser sets it with boundary
    },
    body: formData
  });

  return response.json();
}
```

---

10. Error Handling

HTTP Status Codes

Code	Meaning	Action
200	Success	Process response
400	Bad Request	Fix request data
401	Unauthorized	Refresh token or re-login
409	Conflict	Version mismatch - refresh and retry
413	Payload Too Large	Reduce file size (max 1 MB)
422	Validation Error	Fix field errors
500	Server Error	Retry with exponential backoff

Error Response Format

```
{
  "code": 422,
  "message": "Compliance validation failed",
  "errors": [
    { "step": "owners[0]", "field": "full_name", "message": "This
        field is required" },
    { "step": "owners[0]", "field": "id_document_type", "message":
        "Invalid value. Allowed: nin, bvn, international_passport,
        drivers_license" }
  ]
}
```

Common Error Codes

Code	Description
`VALIDATION_FAILED`	Field validation errors
`VERSION_CONFLICT`	X-Expected-Version doesn't match
`IDEMPOTENCY_CONFLICT`	Same key, different request body
`FILE_TOO_LARGE`	File exceeds 1 MB limit
`UNSUPPORTED_FILE_TYPE`	File type not allowed
`ALREADY_SUBMITTED`	Cannot modify submitted compliance

Appendix: Field Constraints

Field	Max Length	Notes
country	3	ISO code (e.g., "NG")
trading_name	200	
business_description	500	
email	255	Valid email
phone	20	Include country code
address	500	
state	100	Nigerian state (e.g., "Lagos")
lga	100	Local Government Area
full_name	200	
date_of_birth	10	Format: DD-MM-YYYY
registration_number	50	RC number
tax_identification_number	50	TIN
account_number	10	Exactly 10 digits

---

11. FAQ (Frontend Integration Questions)

Q1: Do I need to re-upload files when editing owners/shareholders?

Yes, currently files must be re-uploaded when editing owners or shareholders.

The current implementation replaces all owner/shareholder records on each save. This means:

- When you call `PUT /compliance/owners` or `PUT /compliance/business/shareholders`, all existing records are deleted and replaced with the new submission
- Previously uploaded documents are not preserved
- You must re-submit all files, even for owners/shareholders that haven't changed

Frontend recommendation: When building the edit flow, cache the file objects in state so they can be re-submitted. Alternatively, inform users that editing requires re-uploading all documents.

> ⚠️ **Known limitation**: This behavior is planned for improvement in a future release to preserve documents for unchanged records.

Q2: What happens to document positions when shareholders are removed?

Files are matched by array index, so removing a shareholder shifts all subsequent document associations.

Example scenario: 1. You submit 3 shareholders: Alice (index 0), Bob (index 1), Charlie (index 2) 2. You upload `owner_1_identity` for Bob 3. You edit and remove Bob from the middle 4. Now Charlie becomes index 1 5. If you don't re-upload, Charlie gets Bob's old document

Current behavior: This is a known limitation. Since files are matched by `owner_{index}_*` naming, the backend cannot distinguish between "same person, different position" and "different person, same position."

Frontend recommendation: Always re-upload all documents when the shareholder list changes. Display a warning to users that removing or reordering shareholders requires re-uploading documents for all remaining shareholders.

Q3: What are the valid values for `id_document_type`?

Valid values: - `nin` - National Identification Number - `bvn` - Bank Verification Number - `international_passport` - International Passport - `drivers_license` - Driver's License

Invalid values (do not use): - ~~`national_id`~~ - Removed (was duplicate of `nin`)

The schema in both `gateway-merchant` and `tms-kyc` has been synchronized to accept these exact values.

Q4: How do I view or download previously uploaded documents?

Use the document viewing endpoint:

```
GET /compliance/documents/{document_id}
Authorization: Bearer <token>
```

Where to get `document_id`: The document IDs are returned in the compliance response for each owner/shareholder:

```
{
  "data": {
    "owners": [
      {
        "full_name": "John Doe",
        "id_document_id": "doc_abc123",           // ← Use this ID
        "proof_of_address_document_id": "doc_xyz789"  // ← Or this one
      }
    ]
  }
}
```

Response: Returns the document file as a download (with appropriate Content-Type and Content-Disposition headers).

Q5: How do I detect if compliance has already been submitted?

Check the `status` field in the compliance response:

```
GET /compliance
Authorization: Bearer <token>
```

Response includes status:

```
{
  "data": {
    "status": "draft",      // or "submitted", "approved", "rejected"
    "version": 5
  }
}
```

Status values: | Status | Meaning | Can Edit? | |— ——|— — —|— — —–| | `draft` | Not yet submitted | ✅ Yes | | `submitted` | Under review | ❌ No | | `approved` | Compliance approved | ❌ No | | `rejected` | Returned for corrections | ✅ Yes (must re-submit) |

If you try to edit a submitted compliance:

```
{
  "code": 422,
```

```
  "message": "Compliance has already been submitted and cannot be
        modified"
}
```

Frontend recommendation: 1. Fetch compliance status on page load 2. If `status !== "draft"` and `status !== "rejected"`, disable all form fields 3. Show appropriate messaging based on status

---

Q6: What reference-data endpoints exist, and what about the email field?

Reference data endpoint:

```
GET /compliance/reference-data/{kind}
Authorization: Bearer <token>
```

Available kinds: | Kind | Description | |— —|— — — —-| | `countries` | List of supported countries | | `industries` | Industry categories | | `categories` | Business categories (may depend on industry) | | `states` | Nigerian states | | `lgas` | Local Government Areas (may depend on state) | | `banks` | Supported banks with codes |

About the email field:

The `email` field at the contact level is **only required for starter businesses**.

Business Type	Email Field
Starter	Required in `/compliance/contact`
Registered	Not required (uses support_email and dispute_email)

For registered businesses, the business contact information is captured in the `/compliance/business/details` endpoint, which includes `support_email` and `dispute_email` but not a primary `email` field.

---

12. Completion Checks

How the API determines step completion

The API tracks completion status for each step. Here's what each check validates:

Step	Business Type	Completion Check
Profile	Both	All required profile fields present
Contact	Starter	Email, phone, address, state, lga present
Contact	Registered	Checked as part of business details
Business Details	Registered	Profile + contact fields + business address document
Registration	Registered	RC number and TIN present

Step	Business Type	Completion Check
Owners	Both	At least one owner with `full_name` + required documents
Bank Accounts	Both	At least one bank account with all required fields
Agreement	Both	`agreement_accepted: true` submitted

Completion response format

Each step endpoint returns completion flags:

```
{
  "data": {
    "version": 4,
    "profile_complete": true,
    "contact_complete": true,
    "registration_complete": true,    // Registered only
    "owners_complete": true,
    "bank_accounts_complete": true,
    "agreement_complete": false
  }
}
```

End of Document